Large Reasoning Models

Reasoning, Prompting Techniques, and Instruction Tuning

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 6, 2026

Table of Contents

Motivation

- **Why study LRMs?**
  - LLMs like GPT-3 and GPT-4 excel at generating text.
  - However, they often rely on **pattern matching**, not true reasoning.
  - Real-world applications like **math problem-solving**, **code synthesis**, and **scientific hypothesis generation** require multi-step reasoning.
- **Examples:**
  - *The same model can answer a multi-step problem far better when asked to work through it: appending "Let's think step by step" raises InstructGPT from 17.7% to 78.7% on MultiArith. The knowledge was already in the weights; only the prompt changed.*
  - *DeepMind's AlphaCode reached competition-level programming not by reasoning its way to one answer, but by sampling a very large number of candidate programs and filtering them against the example tests, averaging the top 54.3% of human entrants on Codeforces.*

Learning Outcomes

By the end of this session, you should be able to:

- Define and distinguish Large Reasoning Models (LRMs) from simpler pattern-matching LLMs.
- Name the kinds of reasoning an LRM is asked to perform, and recognise which one a given task calls for.
- Understand advanced reasoning techniques, including Chain-of-Thought (CoT), Tree-of-Thought (ToT), and self-consistency methods.
- Explain how instruction tuning and reinforcement learning (RL) can refine and enhance reasoning abilities in language models.
- Critically assess the limits of current reasoning models, including whether a reasoning trace can be read as an account of how the answer was reached.

What Are Large Reasoning Models (LRMs)?

- **Definition:**
  - Large Language Models (LLMs) optimized for **multi-step logical reasoning**
  - Emphasize Chain-of-Thought (CoT) style reasoning
- **Key Examples:**
  - OpenAI: o1, o3
  - Google: Gemini 2.0 Flash Thinking
  - DeepSeek: R1
  - QwQ (Alibaba)
  - Claude 3.7 Sonnet (extended thinking)
  - Magistral (Mistral)

What Are Large Reasoning Models (LRMs)? (cont.)

- **Distinctive Behavior:**
  - "Thinking" before answering rather than emitting the answer directly
  - Intermediate reasoning traces, either shown to the user or kept hidden
  - Step-by-step intermediate computations
  - More decode steps spent on a hard question than an easy one
  - Better performance on symbolic tasks and math
  - Often paired with advanced prompting or finetuning

What Are Large Reasoning Models (LRMs)? (cont.)

- **Examples:**
  - **PaLM + CoT Prompting:**
    Chain-of-Thought prompting raised PaLM 540B on GSM8K (grade-school math) from 17.9% to 56.9%, without changing a single weight.
  - **Gemini 2.0 Flash Thinking:**
    Exposes the reasoning steps behind an answer instead of hiding them.
  - **DeepSeek R1:**
    Trained with large-scale reinforcement learning on problems whose answers can be checked automatically.
    Its predecessor R1-Zero developed an "aha moment" during training: it began pausing mid-solution to re-examine its own approach, without ever being shown an example of doing so.

LLMs vs. LRMs

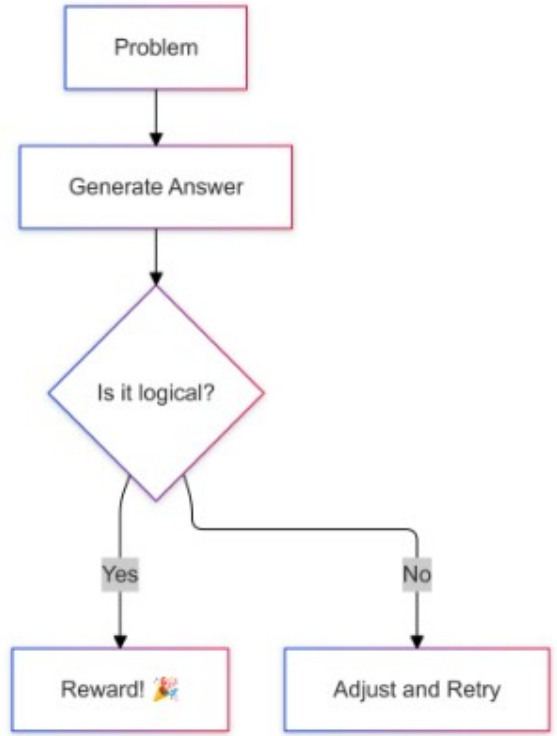
Large language models (LLMs) and large reasoning models (LRMs) share foundational technologies but serve different purposes:

- **Core Function:**
  - **LLMs:** Learn patterns in data to generate fluent, human-like text.
  - **LRMs:** Extend LLMs to solve problems requiring logical reasoning and contextual understanding.
- **Use Cases:**
  - **LLMs:** Content generation, language translation, summarization, and answering user queries.
  - **LRMs:** Math problem solving, interpreting ambiguous clinical data, and complex decision-making.

LLMs vs. LRMs (cont.)

- **Reasoning Ability:**
  - **LLMs:** Limited structured reasoning; may struggle with multi-step logic or ambiguity.
  - **LRMs:** Trained to apply consistent reasoning steps; outputs are more logical and verifiable.
- **Performance:**
  - **LLMs:** Fast response times; optimized for scalability and speed.
  - **LRMs:** Slower response times; require more processing to reason through problems step by step.
- **Best Suited For:**
  - **LLMs:** High-volume, low-risk tasks where speed is critical.
  - **LRMs:** High-stakes or complex tasks where accuracy, explainability, and logic matter.

How Do LRMs Work?



The reward loop behind the RL stage below: a candidate answer is checked, rewarded when the reasoning holds up, and revised when it does not. Source: Chaddha (2025), "Large Reasoning Models: The AI That Solves Problems Like a Human (But 10x Faster)," Medium; the author's own illustrative diagram, not from a paper.

How Do LRMs Work? (cont.)

LRMs use a combination of training methods and prompt strategies to enhance the reasoning capabilities of LLMs.

- **Training on Enriched Datasets**
  - Includes reasoning-focused examples
  - Real-world scenarios with clear outcomes
  - Teaches both correct answers and reasoning steps
- **Reinforcement Learning (RL)**
  - Rewards for logical, correct answers
  - Penalties for incorrect or inconsistent reasoning
  - Reinforces desirable reasoning patterns
- **Human Feedback (RLHF)**
  - Human reviewers guide model outputs

How Do LRMs Work? (cont.)

- Refines nuanced reasoning strategies
- Useful in domains needing expertise (e.g., healthcare, finance)

- **Prompt Engineering**
  - Carefully designed prompts activate reasoning
  - Guides model through multi-step or layered tasks
  - Generic prompts may not trigger deep reasoning
- **Chain-of-Thought (CoT) Prompting**
  - Breaks problems into smaller steps
  - Explores multiple reasoning paths
  - Improves transparency and accuracy
  - Essential for auditability and explainability

Types of Reasoning in LRMs

Large reasoning models (LRMs) employ several types of reasoning to simulate or support human-like problem solving:

- **Deductive Reasoning**
  - Applies general rules to specific cases for logically certain conclusions (top-down reasoning).
  - Excels at structured, rule-based tasks requiring high logical consistency.
  - Example domains: regulatory compliance, medical protocol analysis.
  - *Example:*
    Rule: All patients with a fever and cough should be tested for influenza.
    Case: Patient A has a fever and cough.
    Deduction: Patient A should be tested for influenza.

Types of Reasoning in LRMs (cont.)

- **Inductive Reasoning**
  - Draws general conclusions from specific observations in data.
  - Supports probabilistic predictions and generalization to unseen inputs.
  - Useful in dynamic, data-rich scenarios (e.g., fraud detection).
  - *Example:*
    Observation: 90% of emails with suspicious links are phishing attempts.
    Induction: An email with a suspicious link is likely to be a phishing attempt.

Types of Reasoning in LRMs (cont.)

- **Abductive Reasoning**
  - Infers the most likely explanation from incomplete or uncertain evidence.
  - Generates hypotheses rather than definitive answers.
  - Effective in domains like medical diagnostics, where plausible interpretations are needed.
  - *Example:*
    Evidence: The lawn is wet in the morning.
    Possible explanations: It rained overnight, or the sprinkler was on.
    Abduction: If the weather report says no rain, the most likely explanation is the sprinkler was on.

Types of Reasoning in LRMs (cont.)

- **Analogical Reasoning**
  - Identifies similarities between different situations or datasets.
  - Transfers insights from one context to another for novel problem solving.
  - Example: recommending products based on similar customer behavior.
  - *Example:*
    Situation A: A student learns to solve quadratic equations by factoring.
    Situation B: The student applies the same factoring method to solve cubic equations.
    Analogy: The approach used for quadratics is adapted to cubics.

Reasoning vs. Pattern-Matching in LLMs

- **Pattern-Matching Models**
  - **How it works:** LLMs are trained on vast amounts of text data to learn statistical associations between words and phrases.
  - **Characteristics:**
    - Generate text based on learned patterns without deeper understanding.
    - Often produce fluent and coherent text.
    - May struggle with complex reasoning tasks or multi-step logic.
  - **Example:** Given the prompt "The capital of France is", the model outputs "Paris" by recalling frequent patterns in the training data.
  - **Limitations:**
    - May produce plausible-sounding but incorrect answers if the pattern is misleading.
    - Struggles with tasks that require connecting multiple facts or reasoning through steps.

Reasoning vs. Pattern-Matching in LLMs (cont.)

- **Reasoning-Focused Models**
  - **How it works:** LRMs extend LLMs by incorporating structured reasoning techniques.
  - **Characteristics:**
    - Generate structured intermediate steps to arrive at conclusions.
    - Use multi-hop inference to connect information across several statements or facts.
    - Can break down complex problems into smaller, logical steps.
  - **Example:** To answer "If Alice has 3 apples and buys 2 more, how many does she have?", the model first adds 3 and 2, then states the answer.
  - **Benefits:**
    - Provides explanations or shows their thought process, making answers more transparent.
    - Better at tasks that require logic, calculation, or combining information from different sources.

Reasoning vs. Pattern-Matching in LLMs (cont.)

- **Examples**
  - "Which is heavier: a kilogram of feathers or a kilogram of iron?"

    A pattern-matching model may be misled by the association of "iron" with heaviness.

    A reasoning model will recognize that both weigh the same.

  - In logic puzzles, pattern-based models tend to contradict earlier premises

    Reasoning-aware models (e.g., using Chain-of-Thought prompting) remain consistent and logical.

Reasoning vs. Pattern-Matching in LLMs (cont.)

- **Empirical Separation: When Does Reasoning Matter?**
  - On simple factual questions, both model types may perform well.
  - For challenging tasks like GSM8K (grade-school math), symbolic reasoning, or logic puzzles, pattern-matching alone is not enough.
  - Success on these benchmarks improves dramatically when models are prompted to reason step by step (e.g., using Chain-of-Thought prompting).
  - *Example:* In multi-step math problems, models that show their work (reasoning-focused) achieve much higher accuracy than those that just guess the answer.
  - This highlights the need for explicit reasoning capabilities in advanced language models.

Leading Large Reasoning Models

Following are some of the leading Large Reasoning Models (LRMs) as of 2024/2025:

- **OpenAI o1/o3**
  - **Architecture:** Undisclosed; trained with reinforcement learning to reason before answering.
  - **Training Data Transparency:** Proprietary, limited details.
  - **Strengths:** High-quality reasoning, STEM, coding.
  - **Reasoning Approach:** Hidden chain of thought, with the effort budget selectable as low, medium, or high.
  - **Efficiency:** 200K context window, 100K maximum output tokens.
  - **Best Use Case:** General AI, STEM research, API function calling.

Leading Large Reasoning Models (cont.)

- **DeepSeek R1**
  - **Architecture:** Mixture of Experts (MoE, 671B total, 37B active).
  - **Training Data Transparency:** Open weights (MIT licence); base model pretrained on 14.8T tokens and the RL recipe described in the paper.
  - **Strengths:** Math, coding, and STEM reasoning at open-weight cost.
  - **Reasoning Approach:** Large-scale RL on problems whose answers can be checked automatically, producing long explicit chains of thought.
  - **Efficiency:** Only 37B of 671B parameters are active per token, so each token costs a fraction of a dense model of the same size.
  - **Best Use Case:** Self-hosted reasoning assistant, coding and debugging.

Leading Large Reasoning Models (cont.)

- **Gemini 2.0 (Flash Thinking)**
  - **Architecture:** Multimodal Transformer (Text+Image+Speech).
  - **Training Data Transparency:** Proprietary, multimodal (text, images, APIs).
  - **Strengths:** Multimodal (images, text, speech), deep reasoning.
  - **Reasoning Approach:** Flash Thinking mode (transparent thought process).
  - **Efficiency:** 1M token context (experimental), optimized for speed.
  - **Best Use Case:** AI assistant (multimodal, research, planning).

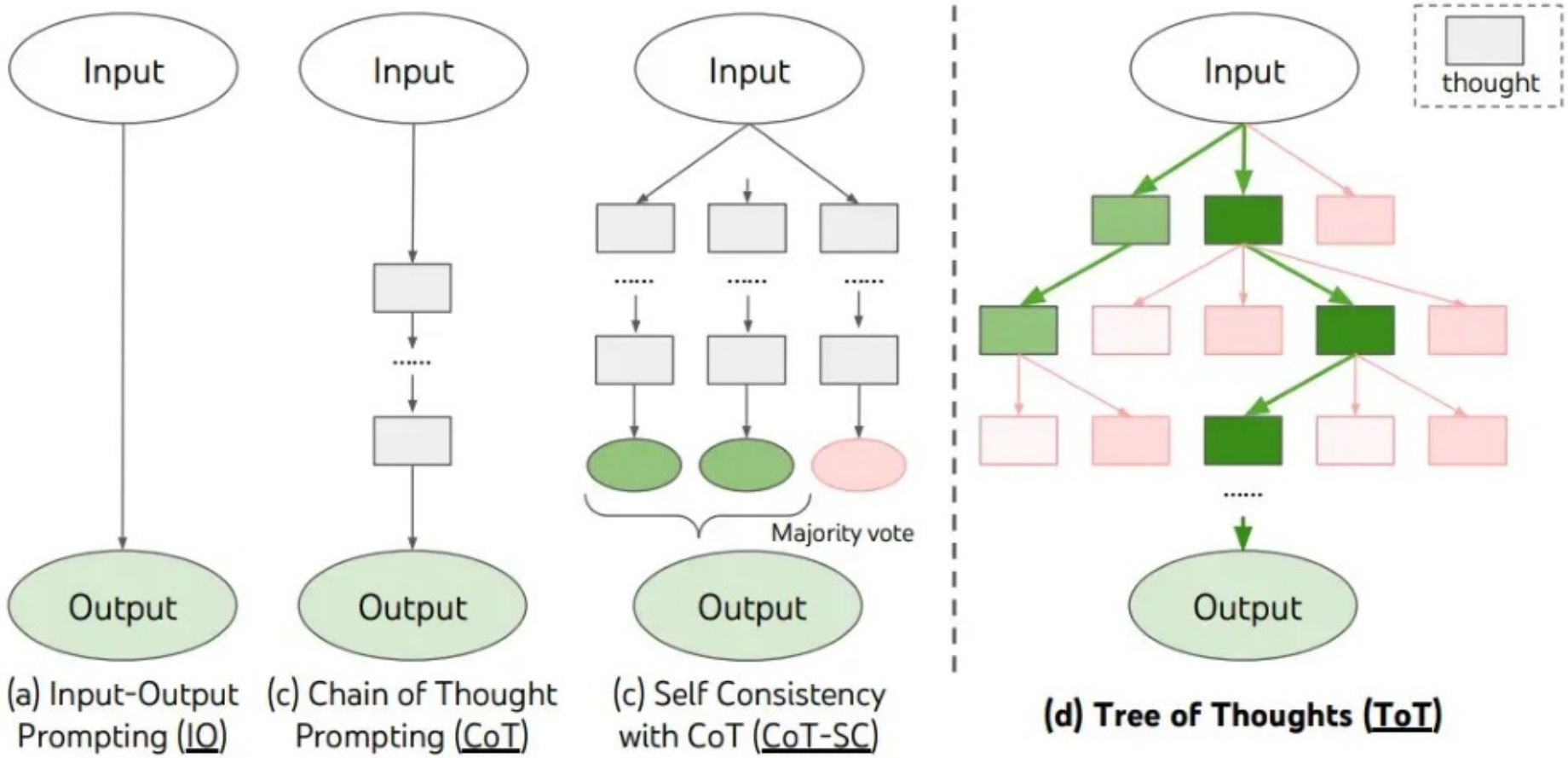
Leading Large Reasoning Models (cont.)

Claude 3.7 Sonnet

- **Architecture:** Undisclosed; a single model that answers either immediately or after extended thinking.
- **Training Data Transparency:** Proprietary; evaluations documented in a published system card.
- **Strengths:** Coding, agentic tasks, long-form dialogue.
- **Reasoning Approach:** Extended thinking, shown to the user rather than hidden.
- **Efficiency:** 200K context; thinking can be switched off, and its token budget set per API call.
- **Best Use Case:** One deployment covering both quick replies and hard problems.

Leading Large Reasoning Models (cont.)

- **QwQ (Alibaba)**
  - **Architecture:** Dense Transformer (32B), RL-trained for reasoning.
  - **Training Data Transparency:** Open weights under Apache 2.0; training corpus undisclosed.
  - **Strengths:** Math, logical puzzles, chain-of-thought self-verification.
  - **Reasoning Approach:** Outcome-reward RL scaled up from a cold-start checkpoint, plus self-reflection.
  - **Efficiency:** 32B parameters, yet comparable to DeepSeek-R1 on several reasoning benchmarks.
  - **Best Use Case:** Mathematical proofs, advanced reasoning on modest hardware.



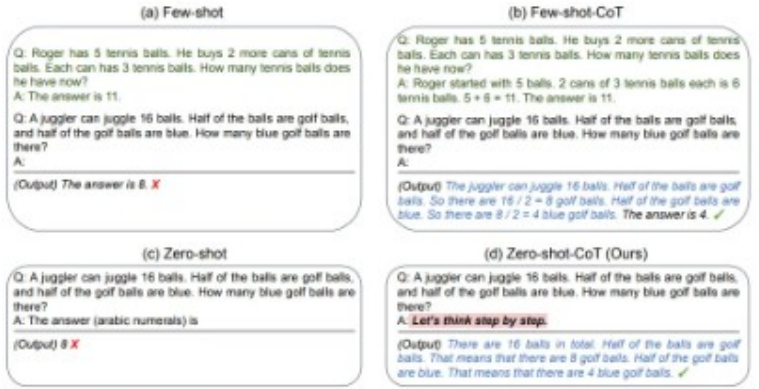
Input-output prompting, Chain-of-Thought, Self-Consistency and Tree-of-Thought side by side.
Source: Yao et al. (2023), Fig. 1.

Chain-of-Thought Prompting (Recap)

- CoT prompting makes the model generate intermediate reasoning steps before the final answer, either by showing worked examples or by appending "Let's think step by step".
- For reasoning models this is the central mechanism: the techniques that follow make the intermediate steps longer, broader, or trained into the weights.
- Beyond basic CoT: automatic exemplar construction (Auto-CoT), search over reasoning paths (Tree-of-Thought), and sampling-based answer selection (Self-Consistency).

Few-shot vs Zero-shot CoT

- Few-shot CoT provides worked exemplars in the prompt (e.g. "If 6 times 4 equals 24, then what is 6 times 5?" followed by its reasoning steps); zero-shot CoT only appends an instruction such as "Let's think step by step".
- Few-shot buys accuracy at the cost of hand-crafting exemplars for each task; zero-shot avoids that cost but is weaker on hard problems. Auto-CoT aims for few-shot accuracy without the manual effort.

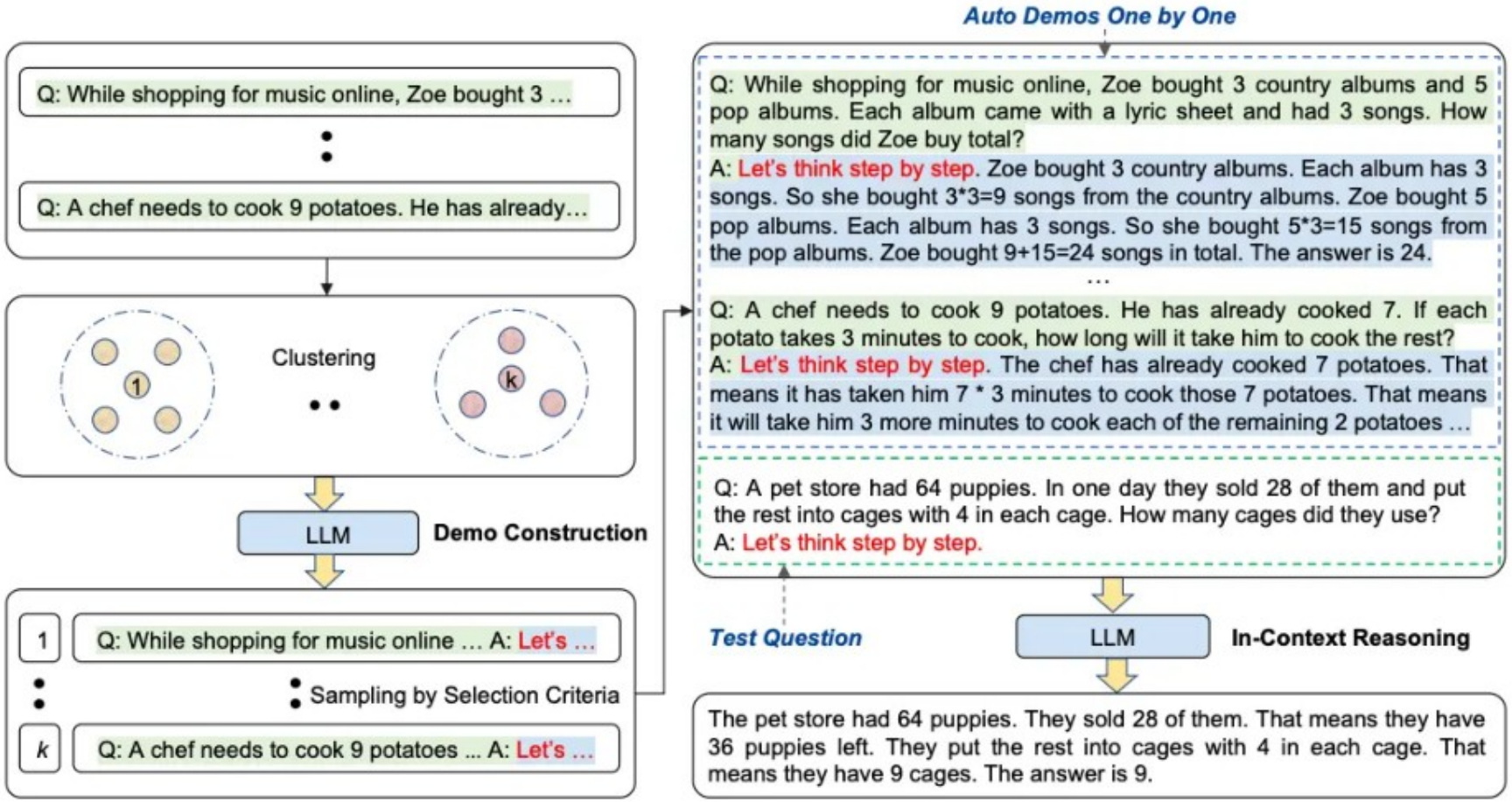


Four prompting settings on the same question: (a) Few-shot, (b) Few-shot-CoT, (c) Zero-shot and (d) Zero-shot-CoT. Only the two CoT settings reach the right answer. Source: Kojima et al. (2022), Fig. 1.

Auto-CoT Prompting (cont.)

- **What is Auto-CoT?**
  - A method for building the worked exemplars that few-shot CoT needs, without writing them by hand.
  - The reasoning chains in the prompt are generated by the model itself rather than by an annotator.
- **How it works:**
  - **Cluster:** encode the task's questions with Sentence-BERT and partition them into k clusters.
  - **Construct:** from each cluster take the question closest to the centre and generate its chain with Zero-shot CoT ("Let's think step by step"), keeping the first chain that passes simple length and format criteria.
  - The k resulting question-and-chain pairs become the few-shot prompt. Clustering spreads the exemplars over different question types, so one flawed chain cannot dominate the prompt.

Auto-CoT Prompting (cont.)



Auto-CoT Prompting. Source: Zhang et al. (2022), Fig. 4.

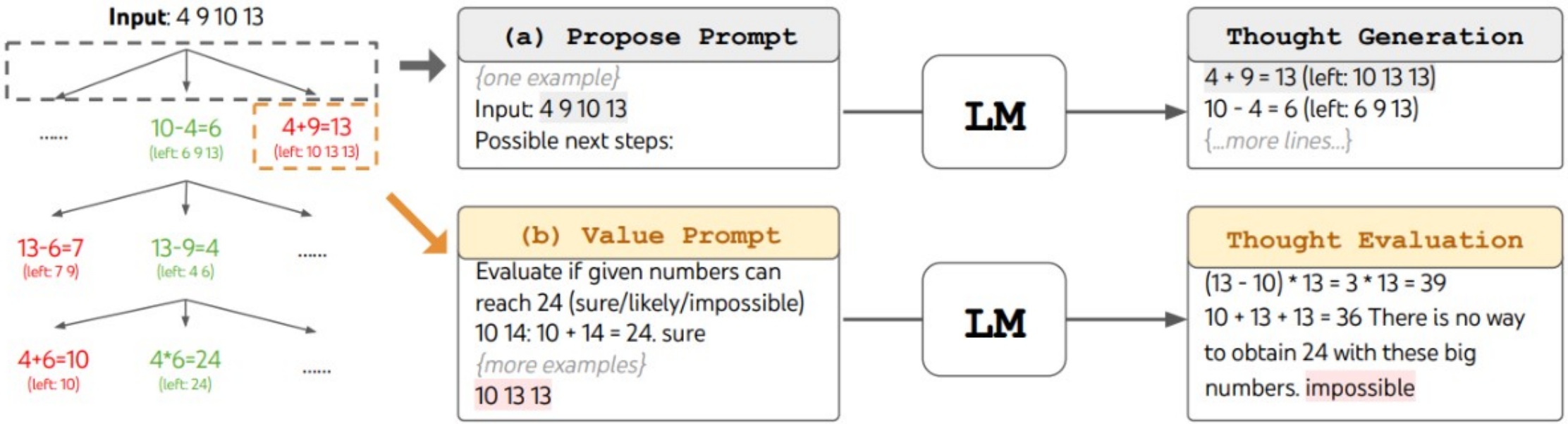
Tree-of-Thought (ToT) Reasoning (cont.)

- **What is ToT?**
  - A reasoning technique that structures the thought process as a tree, allowing for exploration of multiple reasoning paths.
  - Each node in the tree represents a step in the reasoning process, and branches represent different possible paths or decisions.
- **How it works:**
  - The model generates a tree structure where each branch represents a different reasoning path.
  - The model can explore multiple paths simultaneously, evaluating the outcomes of each.

Tree-of-Thought (ToT) Reasoning (cont.)

- **Benefits:**
  - Allows for more complex reasoning by considering multiple possibilities at once.
  - Helps the model avoid getting stuck in a single line of reasoning, improving problem-solving capabilities.
- **Example (Game of 24):**
  - Given four numbers, reach 24 using each exactly once. A thought is one intermediate equation, e.g. "$4 + 9 = 13$ (left: 5, 10, 13)".
  - At each level the model proposes candidate next equations, then rates each resulting state as sure, maybe, or impossible, and only the most promising states are expanded.
  - Search matters here because a single greedy chain commits to its first arithmetic step and cannot recover: CoT solves 4% of these puzzles, ToT with breadth 5 solves 74%.

Tree-of-Thought (ToT) Reasoning (cont.)



ToT in a game of 24. The LM is prompted for (a) thought generation and (b) valuation. Source: Yao et al. (2023), Fig. 2.

Tree-of-Thought (ToT) Reasoning (cont.)

Method	Success
IO prompt	7.3%
CoT prompt	4.0%
CoT-SC (k=100)	9.0%
ToT (ours) (b=1)	45%
ToT (ours) (b=5)	**74%**
IO + Refine (k=10)	27%
IO (best of 100)	33%
CoT (best of 100)	49%

Table 2: Game of 24 Results.

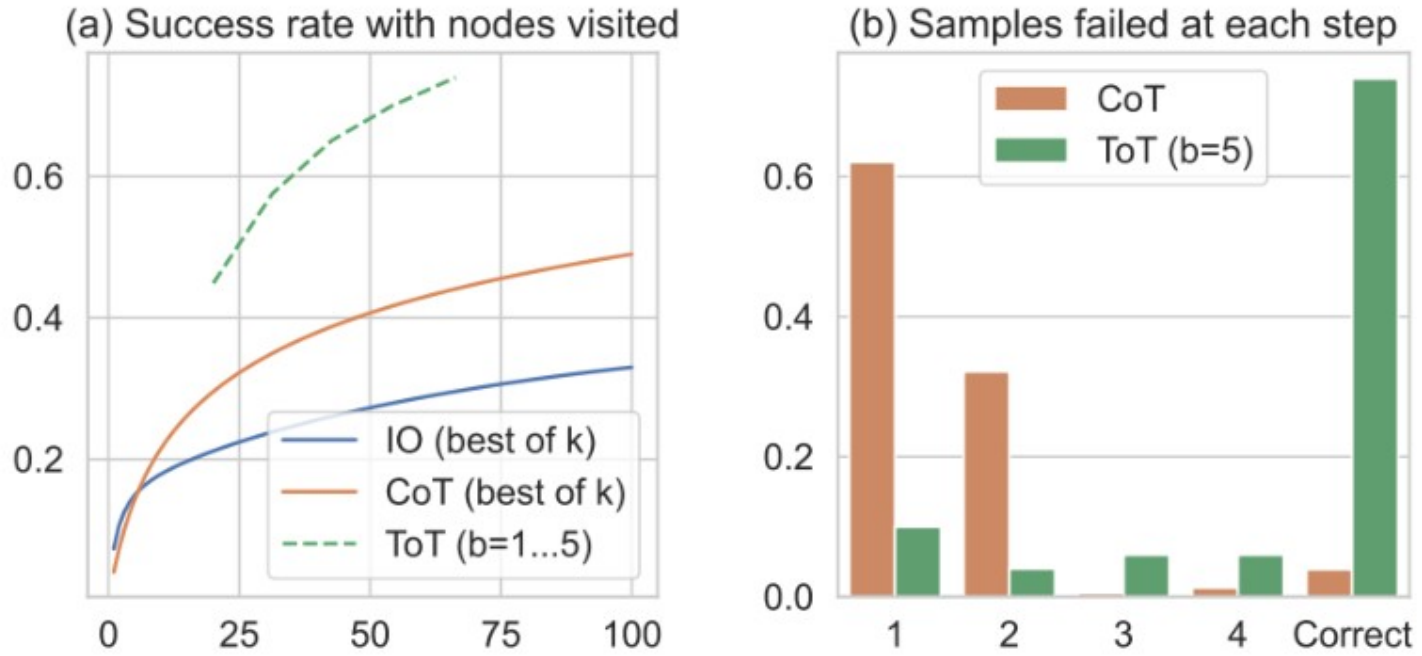


Figure 3: Game of 24 (a) scale analysis & (b) error analysis.

Source: Yao et al. (2023), Table 2 and Fig. 3.

Self-Consistency (SC)

- Self-consistency samples several reasoning paths for the same prompt, then keeps the answer the paths agree on.
- The figures below give the empirical picture from Wang et al. (2022): consistent gains across arithmetic and commonsense benchmarks, growing with the number of sampled paths, at a linear increase in inference cost.
- Sampling many paths at inference time is an early form of test-time compute scaling, the same trade later used by dedicated reasoning models.

Self-Consistency (SC) (cont.)

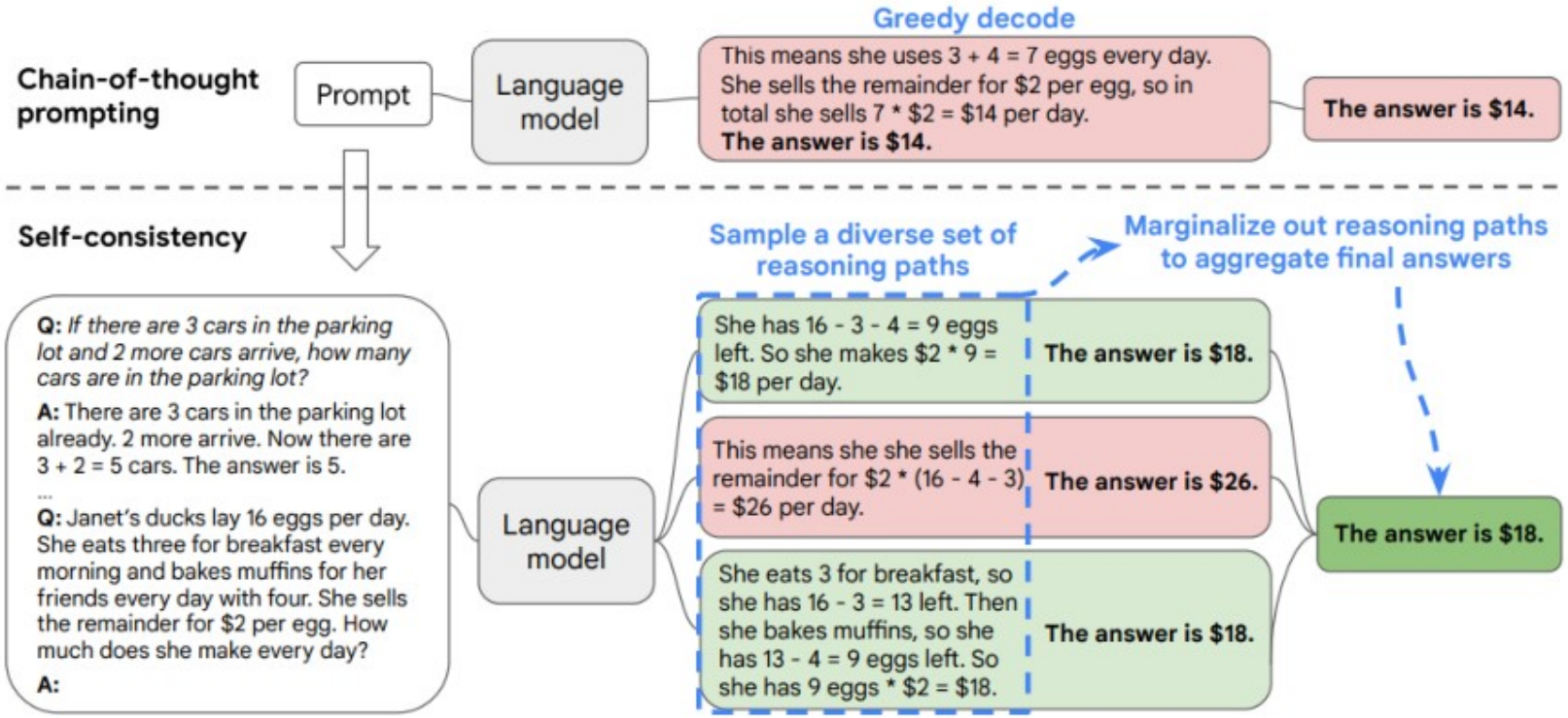


Figure 1: The self-consistency method contains three steps: (1) prompt a language model using chain-of-thought (CoT) prompting; (2) replace the "greedy decode" in CoT prompting by sampling from the language model's decoder to generate a diverse set of reasoning paths; and (3) marginalize out the reasoning paths and aggregate by choosing the most consistent answer in the final answer set.

Source: Wang et al. (2022), Fig. 1.

Self-Consistency (SC) (cont.)

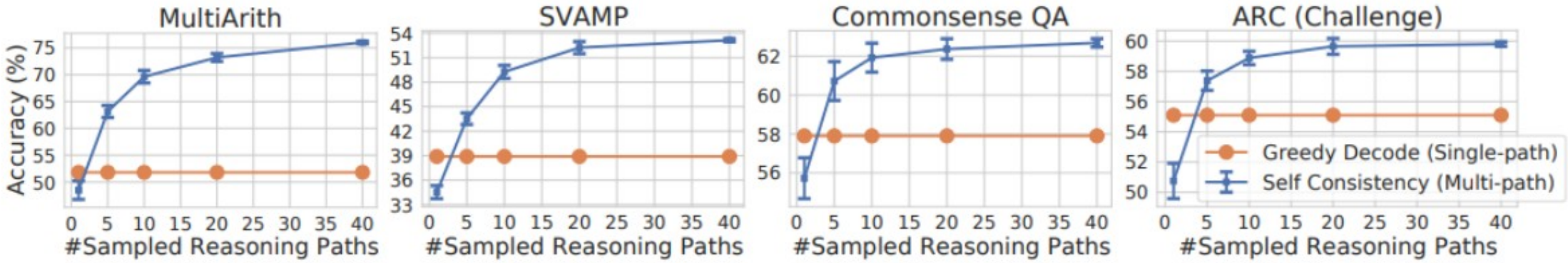
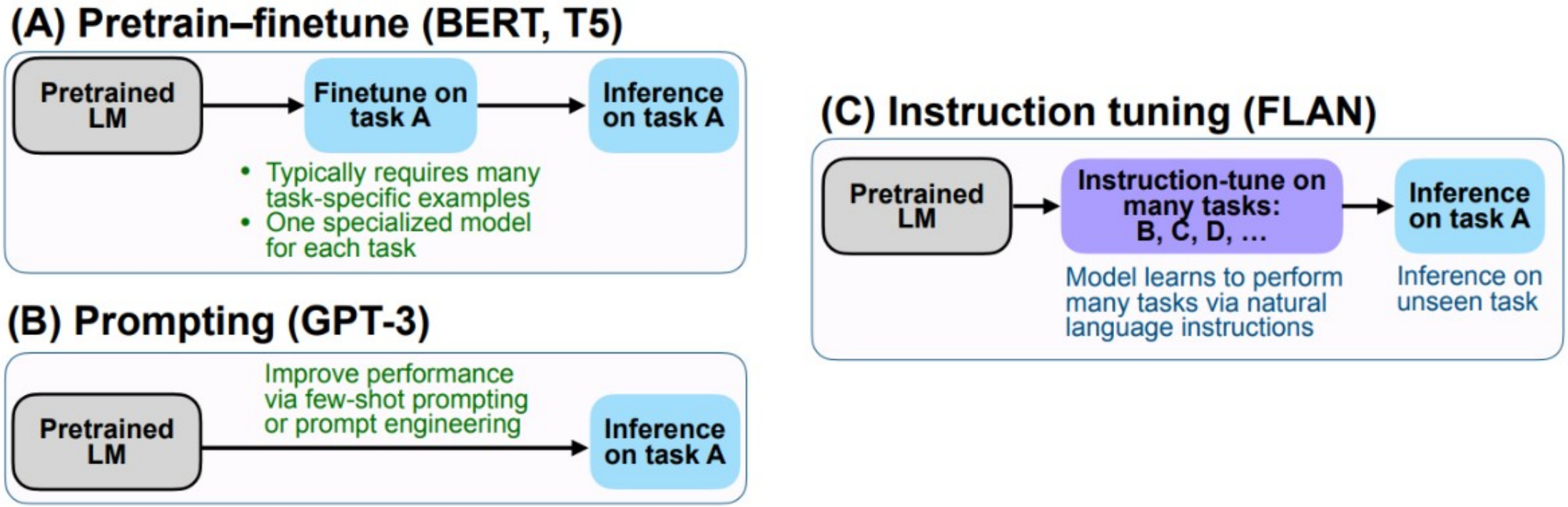


Figure 2: Self-consistency (blue) significantly improves accuracy over CoT-prompting with greedy decoding (orange) across arithmetic and commonsense reasoning tasks, over LaMDA-137B. Sampling a higher number of diverse reasoning paths consistently improves reasoning accuracy.

Source: Wang et al. (2022), Fig. 2. Note that the gain saturates: extra samples buy less and less.



Instruction tuning next to the two paradigms it draws on: task-specific finetuning and few-shot prompting. Source: Wei et al. (2022), Fig. 2.

Instruction Tuning (cont.)

Instruction Tuning vs. Multi-Task Fine-Tuning (FLAN Ablation Study)

- **Research Question:** Are the benefits of instruction tuning due to the instructions themselves, or just from fine-tuning on multiple tasks?
- **Ablation Setups:**
  - *No Template:* Only input-output pairs, e.g., input: "the dog runs", output: "le chien court".
  - *Dataset Name:* Input prepended with task and dataset, e.g., "[Translation: WMT 14 to French] The dog runs".
  - *FLAN Instructions:* Full natural language instruction, e.g., "Please translate this sentence to French: 'The dog runs.' "
- **Results:** zero-shot accuracy averaged over four held-out task clusters.
  - FLAN instructions 55.2%, dataset name 46.6%, no template 37.3%: a gap of 8.6 and 17.9 points respectively.
  - **Conclusion:** Most of the benefit comes from the instructions themselves, not from multi-task fine-tuning alone.

Instruction Tuning (cont.)

Chain-of-Thought (CoT) Fine-Tuning

- **How is it done?** This can be achieved by:
  - Few-shot prompting with exemplars that demonstrate sequential reasoning.
  - Appending phrases like "think step by step" to prompts.
- **Why is it important?**
  - CoT prompting significantly enhances zero-shot performance on arithmetic, symbolic, and logical reasoning tasks.
  - Research (Chung et al., 2022) shows that instruction tuning without CoT tasks degrades performance on CoT evaluations.
  - Including CoT tasks in instruction tuning improves performance across all evaluations.
- **Key Insight:** Fine-tuning on CoT tasks, both with and without few-shot exemplars, enables models to better perform step-by-step reasoning, rather than jumping to linguistically plausible answers.

Instruction Tuning (cont.)

Instruction Tuning a Small Model on a Large One's Reasoning

- FLAN instruction-tunes on human-written task templates. A cheaper route is to let a strong model write the training data: prompt a teacher LLM with Zero-shot CoT, then instruction-tune a small student on the resulting reasoning chains.
- Ranaldi & Freitas call this *Self-refine Instruction-tuning* and add a second phase: the student generates its own answers and is then tuned on preferences over them with Direct Preference Optimization.
- Both phases matter: adding self-refinement significantly outperforms instruction tuning alone, both on the tuned tasks and on held-out ones, bringing the small students closer to the reasoning ability of the much larger teachers.

Limitations of Large Reasoning Models

- **Verbose outputs:** Chain-of-Thought and Tree-of-Thought produce long outputs.
  - Extensive reasoning traces can bury the answer the user actually asked for.
- **Prompt sensitivity:** Small changes in wording lead to large changes in output.
  - CoT lifts GSM8K accuracy substantially, but the size of the gain depends on how the exemplars are written.
  - Careful prompt design is therefore part of the method, not an optional extra.
- **Computation overhead:** Worst with self-consistency sampling and Tree-of-Thought search.
  - Cost grows with the number of sampled paths or expanded nodes, so accuracy is bought with inference time.

Can the Reasoning Trace Be Trusted?

The techniques above were each introduced with the claim that visible reasoning makes a model easier to audit. That claim needs qualifying.

- **A trace need not describe the computation.**
  - Turpin et al. (2023) added a biasing feature to the prompt, for example reordering multiple-choice options so the correct answer was always the same letter.
  - Models followed the bias, wrote fluent reasoning that never mentioned it, and lost up to 36 accuracy points across 13 BIG-Bench Hard tasks.
  - So a trace can rationalise an answer after the fact rather than explain how it was produced. It is evidence about what the model will say, not about what it did.

Can the Reasoning Trace Be Trusted? (cont.)

- **What follows for practice**
  - Check the answer, not the prose: where correctness matters, verify the output against something external such as a unit test, a solver, or a known result.
  - Do not present a trace to a user as an explanation of the model's decision unless it has been tested for faithfulness.
  - Automatic checking of answers is exactly what the reinforcement-learning approach covered later builds on, and it is the reason that approach is more reliable than prompting alone.

Summary

- **Large reasoning models** differ from pattern-matching LLMs: they spend test-time compute on explicit multi-step reasoning traces.
- **Prompting techniques** (Chain-of-Thought, Tree-of-Thought, Self-Consistency) elicit reasoning from capable models without any training.
- **Instruction tuning** (FLAN and successors) teaches models to follow task descriptions and generalize to unseen tasks; CoT fine-tuning bakes reasoning into the weights.
- The leading systems (o1/o3, DeepSeek-R1, Gemini Flash Thinking, QwQ) combine these ingredients with RL post-training.
- Costs remain: verbose traces, prompt sensitivity, and inference compute overhead.
- **A trace is not a proof.** Models can write reasoning that omits what actually drove the answer, so verify the answer rather than trusting the prose.